

MediTrack: A Multi-Stakeholder Mobile Health Platform with On-Device Clinical Analytics and Role-Based Security Architecture

Abstract—Mobile health (mHealth) applications have emerged as critical tools for chronic disease management, yet most existing solutions address only a single stakeholder—typically the patient—leaving fragmented workflows between patients, physicians, pharmacies, and administrators. This paper presents MediTrack, a comprehensive multi-stakeholder Android-based health management platform that unifies medication tracking, vital sign monitoring, clinical decision support, pharmacy inventory management, and real-time doctor-patient communication under a single, security-hardened architecture. MediTrack employs a modular Model-View-ViewModel (MVVM) architecture with Hilt-based dependency injection, backed by Firebase cloud services for real-time data synchronization. A key contribution is the on-device health analytics pipeline, comprising five interconnected engines—RiskScoreEngine, TrendPredictionEngine, VitalAlertEngine, HealthInsightEngine, and UnifiedAlertPrioritizer—that perform composite risk scoring, consecutive-trend detection, threshold-based vital alerting, and personalized health suggestions without requiring server-side computation. The platform enforces a fine-grained role-based access control (RBAC) model with server-side Firestore security rules spanning 540 lines, implementing immutable audit trails, state-machine-validated order workflows, and cross-tenant data isolation. We detail the system architecture, security model, analytics pipeline, and multi-role workflows, and evaluate the platform through quantitative codebase metrics and a comparative feature analysis against existing mHealth applications. MediTrack demonstrates that a single mobile application can effectively serve the complete healthcare ecosystem while maintaining clinical data integrity and privacy through defense-in-depth security.

Index Terms—Clinical decision support, Firebase, health analytics, medication adherence, mobile health (mHealth), MVVM architecture, pharmacy management, role-based access control.

NOMENCLATURE

ADA	American Diabetes Association
AHA/ACC	American Heart Association /

CDSS	American College of Cardiology
DI	Clinical Decision Support System
EHR	Dependency Injection
FHIR	Electronic Health Record
	Fast Healthcare Interoperability
	Resources
JNC	Joint National Committee
mHealth	Mobile Health
MVVM	Model-View-ViewModel
RBAC	Role-Based Access Control
TLS	Transport Layer Security

I. INTRODUCTION

THE proliferation of smartphones has created unprecedented opportunities for healthcare delivery through mobile applications. The global mobile health market, valued at \$68.1 billion in 2024, is projected to reach \$362.7 billion by 2032, driven by rising chronic disease burden, increasing smartphone penetration, and growing demand for patient-centric healthcare solutions [1]. Despite this growth, the mHealth landscape remains fragmented: medication reminder applications lack clinical oversight, electronic health record (EHR) systems provide limited patient engagement, and pharmacy platforms operate in isolation from clinical workflows.

Current mHealth solutions suffer from three fundamental limitations. First, *stakeholder fragmentation*: most applications serve a single user role, forcing healthcare ecosystems to rely on multiple disconnected tools. A patient may use one application for medication reminders, another for health logging, and communicate with their physician through yet another platform. Second, *limited on-device intelligence*: health analytics typically require server-side processing or cloud-based machine learning services, introducing latency, privacy concerns, and connectivity dependencies. Third, *insufficient security architectures*: many mHealth applications rely solely on client-side validation, leaving clinical data vulnerable to unauthorized access and tampering.

This paper presents MediTrack, a comprehensive mobile health platform designed to address these limitations through three principal contributions:

- 1) **Multi-stakeholder architecture**: A unified platform serving four distinct roles—Patient, Doctor, Pharmacy, and Administrator—with role-specific dashboards, workflows, and data access patterns, eliminating the need for multiple disconnected applications.

Manuscript submitted March 2026. This work was conducted as part of academic research in mobile health systems development at Narayana Engineering College, Nellore.

¹Vijaya Wuijwal Reddy, ²Baddipudi Krishna Sai, ³Budamagunta Ravi Teja, and ⁴Nuthalapati Lohith are students in the Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India (e-mail: wuijwalreddy@gmail.com).

⁵K. Divya, M.Tech (Ph.D), is an Assistant Professor in the Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India (Mentor).

- 2) **On-device health analytics pipeline:** A modular analytics engine comprising five interconnected components that perform composite health risk scoring, trend prediction, vital threshold alerting, and personalized health suggestions entirely on the mobile device, enabling real-time clinical intelligence without cloud computation dependencies.
- 3) **Defense-in-depth security model:** A layered security architecture combining server-side Firestore security rules with role-based access control, immutable audit trails, state-machine-validated workflows, and cross-tenant data isolation, ensuring clinical data integrity and compliance with healthcare data protection principles.

The remainder of this paper is organized as follows. Section II reviews related work in mHealth platforms, clinical decision support, and mobile security. Section III presents the overall system architecture. Section IV details the multi-role workflow design. Section V describes the on-device analytics pipeline. Section VI elaborates on the security architecture. Section VII presents the evaluation methodology and results. Section VIII discusses limitations and future work. Section IX concludes the paper.

II. RELATED WORK

A. Mobile Health Platforms

Mobile health applications have evolved from simple medication reminders to comprehensive health management platforms. Medisafe [2] pioneered cloud-synced medication tracking with family monitoring capabilities but lacks clinical decision support and pharmacy integration. MyTherapy [3] combines medication reminders with health journaling and offers healthcare provider reporting; however, it does not support real-time doctor-patient communication or multi-role access control. CareZone [4] integrates medication management with pharmacy features but operates as a patient-only application without clinical decision workflows.

More comprehensive platforms have emerged in the clinical domain. Epic MyChart [5] extends electronic health records to mobile devices with appointment scheduling, messaging, and test results access. However, MyChart functions as a patient portal tethered to specific healthcare institutions rather than an independent platform, and it lacks on-device analytics or pharmacy inventory management. Apple Health [6] and Google Health Connect [7] provide health data aggregation frameworks but do not implement clinical workflows or multi-stakeholder interaction.

B. Clinical Decision Support Systems

Clinical decision support systems (CDSS) have been extensively studied in hospital settings [8]. Mobile CDSS implementations typically fall into two categories: rule-based systems that apply predefined clinical guidelines [9] and machine-learning-based systems that require cloud infrastructure [10]. MediTrack's approach occupies a middle ground: it implements evidence-based clinical rules (blood pressure staging per JNC guidelines, glucose thresholds per ADA standards) in

a modular on-device pipeline that can operate without network connectivity.

Recent work on mobile clinical analytics includes HealthAssist [11], which uses smartphone sensors for activity-based health predictions, and VitalConnect [12], which focuses on continuous vital sign monitoring from wearable devices. Unlike these specialized solutions, MediTrack's analytics pipeline processes user-reported health logs to generate composite risk scores, detect vital sign trends, and produce actionable health suggestions—functionality that does not require specialized hardware.

C. Security in mHealth Applications

Healthcare data security remains a critical concern in mHealth. A systematic review by Iwaya *et al.* [13] found that 80% of surveyed health applications had significant security vulnerabilities, including inadequate access control and unencrypted data storage. The OWASP Mobile Security Testing Guide [14] provides a framework for mobile application security assessment, emphasizing authentication, data storage, network communication, and platform interaction.

Firebase Security Rules, used in MediTrack's backend, provide a declarative security model that is evaluated server-side, preventing client-side bypass [15]. While previous work has documented Firebase security patterns for general-purpose applications [16], this paper demonstrates their application to healthcare-specific requirements: immutable audit trails, state-machine-validated clinical workflows, and multi-tenant pharmacy data isolation.

D. Comparison with Existing Solutions

Table I summarizes the feature comparison between MediTrack and existing mHealth platforms, highlighting MediTrack's unique combination of multi-stakeholder support, on-device analytics, and comprehensive security architecture.

III. SYSTEM ARCHITECTURE

A. Architectural Overview

MediTrack follows the Model-View-ViewModel (MVVM) architectural pattern, a well-established design for Android applications that promotes separation of concerns, testability, and lifecycle awareness. Fig. 1 illustrates the high-level system architecture, comprising four principal layers: Presentation, Domain, Data, and Infrastructure.

The Presentation Layer consists of 33 Activity classes utilizing ViewBinding for type-safe view references, driven by 22 ViewModels that expose UI state through LiveData observables. The Domain Layer houses use cases that orchestrate complex multi-repository operations. The Data Layer comprises 14 repositories serving as the single source of truth, bridging ViewModels to the Infrastructure Layer. The Infrastructure Layer is built upon Firebase services including Cloud Firestore (17 collections), Firebase Authentication, and Firebase Storage.

TABLE I
FEATURE COMPARISON OF MHEALTH PLATFORMS

Feature	Medisafe	MyTherapy	MyChart	Apple H.	CareZone	MediTrack
Medication Tracking	Yes	Yes	Yes	No	Yes	Yes
Health Vital Logging	No	Limited	Yes	Yes	No	Yes
Clinical Decision Support	No	No	Limited	No	No	Yes
Multi-Role RBAC	No	No	Yes*	No	No	Yes
On-Device Analytics	No	No	No	Limited	No	Yes
Doctor-Patient Chat	No	No	Yes	No	No	Yes
Pharmacy Integration	No	No	No	No	Limited	Yes
Appointment Mgmt.	No	No	Yes	No	No	Yes
Prescription Versioning	No	No	Yes	No	No	Yes
Offline Support	Limited	Limited	No	Yes	No	Yes
Immutable Audit Trails	No	No	Yes*	No	No	Yes
Risk Score Computation	No	No	No	No	No	Yes

*Within institutional deployment only.

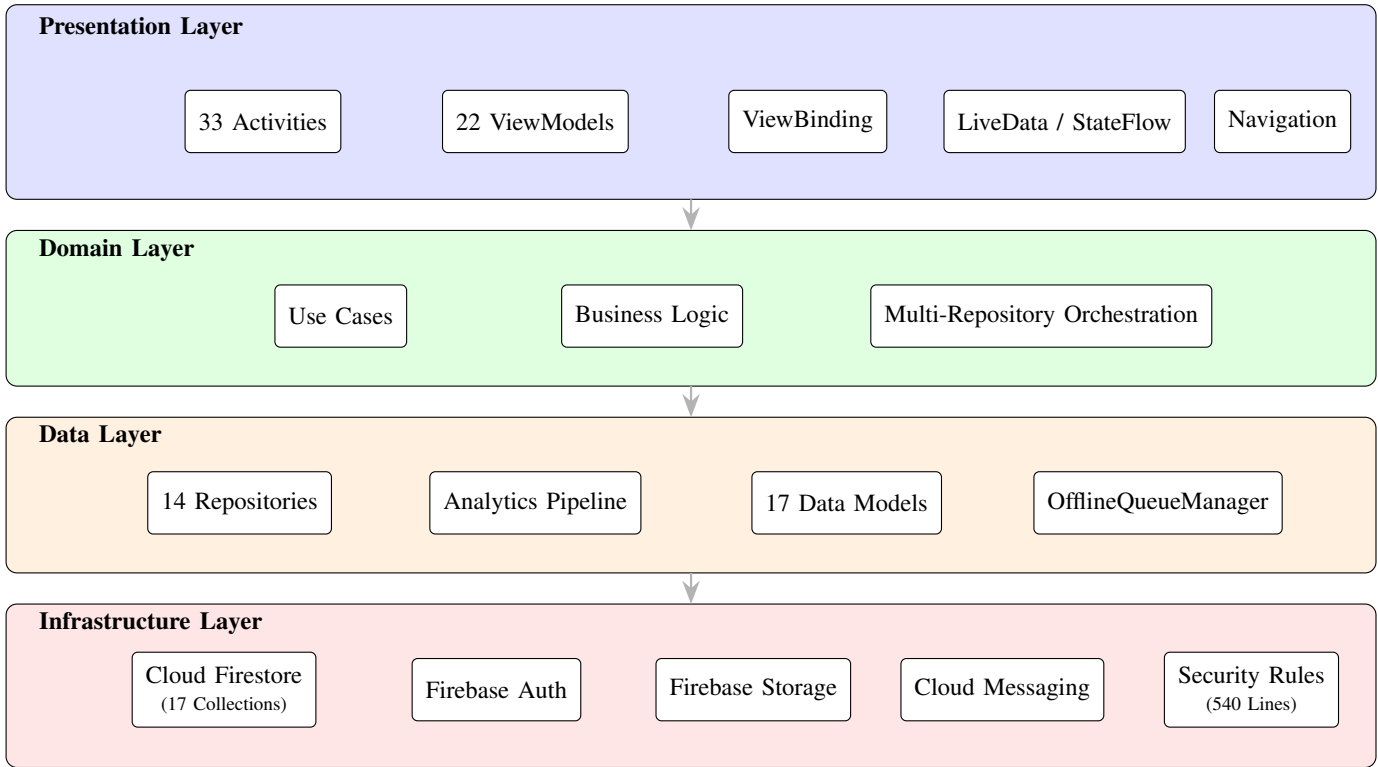


Fig. 1. MediTrack system architecture showing the four-layer MVVM design with Firebase infrastructure. The Presentation Layer contains 33 Activities driven by 22 ViewModels. The Domain Layer houses use cases for complex multi-repository orchestration. The Data Layer comprises 14 repositories and the analytics pipeline. The Infrastructure Layer is built on Firebase services with 540-line server-side security rules.

B. Technology Stack

MediTrack is developed in Kotlin targeting Android SDK 36 (minimum SDK 24, covering 98.1% of active devices). Table II summarizes the key technologies employed.

C. Data Model

MediTrack's data layer comprises 17 Firestore collections storing 17 distinct model types. The data model was designed with three principles: (i) denormalization for read performance, following Firestore best practices; (ii) immutability for audit-critical collections; and (iii) role-scoped access patterns. Fig. 2 presents the entity-relationship diagram for the core data model.

Key design decisions include:

- **Prescriptions are never deleted:** The prescriptions collection enforces `allow delete: if false` at the Firestore rules level, preserving a complete audit trail of all medical prescriptions.
- **Medicine intakes are immutable:** Once a patient records taking a medication, the record cannot be modified or deleted, ensuring adherence data integrity.
- **Multi-doctor support:** Users maintain an `assignedDoctors` array, enabling multiple physicians to access a patient's records—reflecting the reality that patients often see multiple specialists.

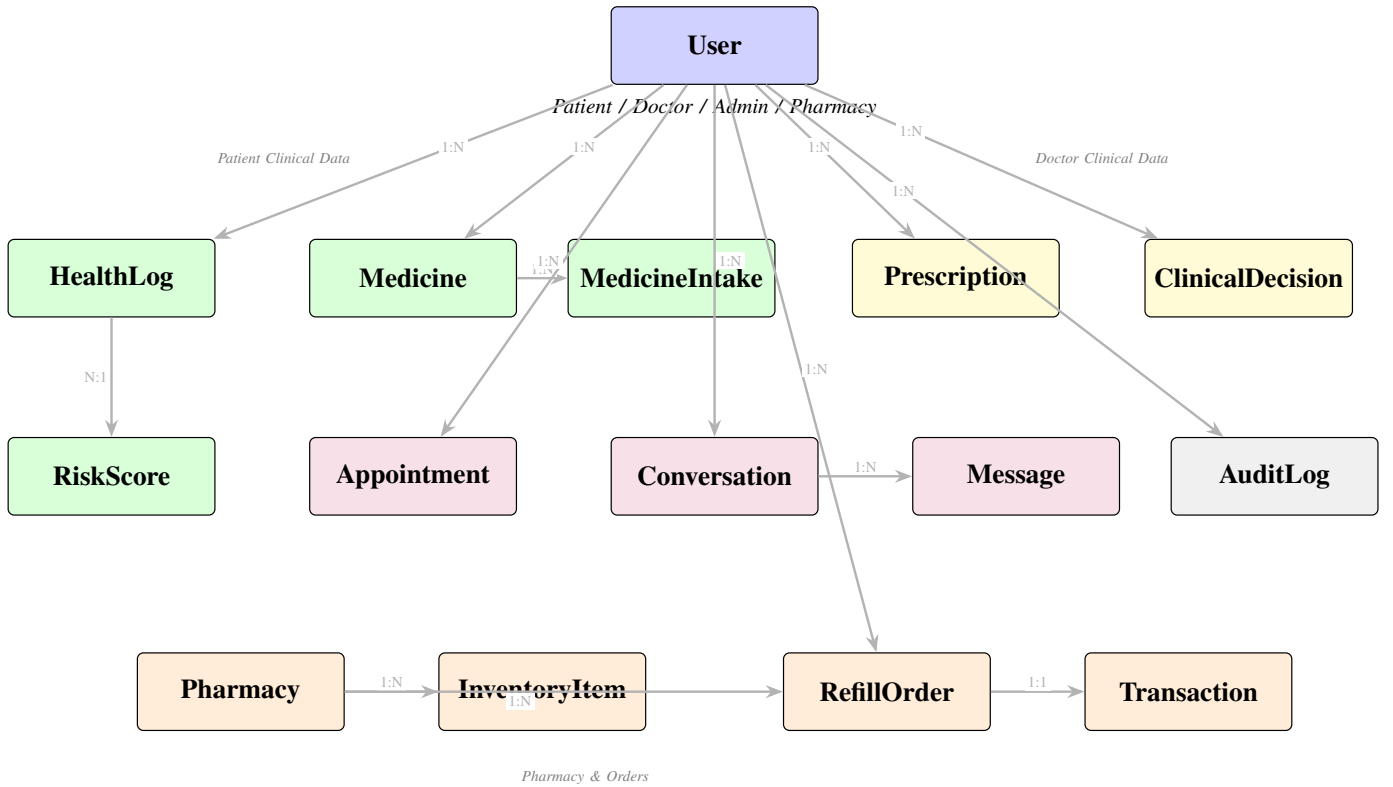


Fig. 2. Entity-relationship diagram of MediTrack's core data model showing relationships between Users (Patient, Doctor), HealthLogs, Medicines, ClinicalDecisions, PrescriptionRecords, Appointments, RefillOrders, Pharmacies, InventoryItems, Conversations, and Messages across 17 Firestore collections.

TABLE II
TECHNOLOGY STACK

Component	Technology	Purpose
Language Arch.	Kotlin 2.1.0 MVVM + Clean	Development lang. Separation of concerns
DI	Hilt (Dagger) + KSP	Compile-time DI
Backend Async	Firebase Suite Coroutines + Flow	Cloud data & auth Reactive async
UI	ViewBinding	Type-safe views
Charts	MPAndroidChart	Data visualization
Background Alarms	WorkManager AlarmManager + Svc.	Periodic sync Med. reminders
Offline Security	PersistentCache Firestore Rules	Offline-first access Server-side RBAC

D. Dependency Injection Architecture

MediTrack employs Hilt, Google's recommended dependency injection framework for Android, configured with two primary modules. `FirestoreModule` provides singleton instances of `FirebaseAuth`, `FirebaseFirestore`, and `FirebaseStorage`, ensuring a single connection pool throughout the application lifecycle. `RepositoryModule` binds all 14 repository implementations as singletons, receiving Firebase instances via constructor injection.

ViewModels are annotated with `@HiltViewModel` and

declare repository dependencies in their constructors, enabling automated injection. This architecture ensures that repositories maintain a single instance throughout the application lifecycle, preventing duplicate Firestore listeners and reducing memory consumption.

IV. MULTI-ROLE WORKFLOW DESIGN

A. Role-Based Navigation

MediTrack implements a four-role system reflecting the primary stakeholders in an outpatient healthcare ecosystem. Upon successful authentication, a `RoleBasedNavigator` component directs users to role-specific entry points based on their Firestore user document: `PATIENT` to `HomeDashboardActivity`, `DOCTOR` to `DoctorDashboardActivity`, `ADMIN` to `AdminDashboardActivity`, and `PHARMACY` to `PharmacyDashboardActivity`. Fig. 3 illustrates this flow.

Doctor and Pharmacy accounts require explicit administrator approval before accessing protected features. Unapproved accounts are directed to `AccountPendingActivity`, and the Firestore security rules enforce this requirement server-side through the `isApproved()` helper function, which explicitly checks for an "APPROVED" status field—missing status values are never treated as approved.

B. Patient Workflows

The patient role encompasses five primary workflows:

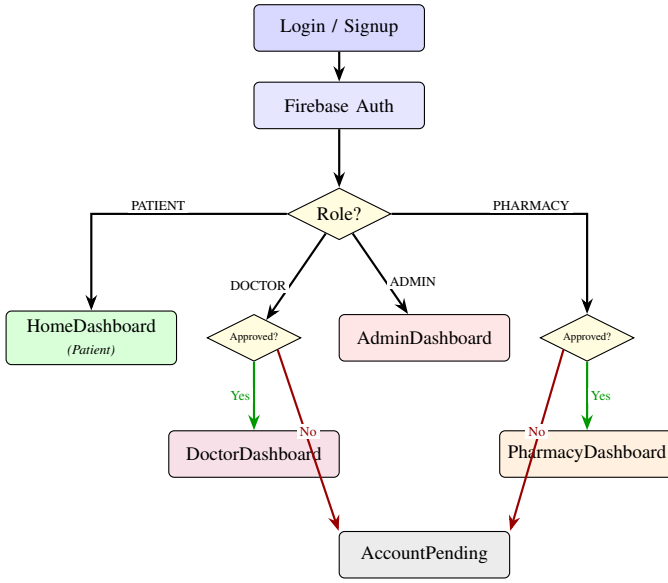


Fig. 3. Role-based navigation flow showing authentication, role detection, approval verification, and role-specific dashboard routing for all four user types. Doctor and Pharmacy roles require explicit admin approval.

1) Medication Management: Patients add medications with name, dosage, frequency, unit, and reminder times. The system schedules exact alarms via Android’s `AlarmManager`, which trigger a chain of `BroadcastReceiver` → `ForegroundService` → `Notification` with full-screen intent. Medication intake is tracked through an immutable `medicineIntakes` collection, and adherence percentage is computed in real-time. A `BootReceiver` reschedules all alarms after device restart.

2) Health Vital Logging: Patients record vital signs including blood pressure (systolic/diastolic), heart rate, glucose level, temperature, weight, and oxygen saturation. Each entry can include symptom tags and free-text notes. Upon entry, the `VitalAlertEngine` evaluates readings against clinical thresholds and generates immediate alerts for abnormal values.

3) Appointment Booking: Patients view available doctor time slots, book appointments (consultation, follow-up, emergency, or routine checkup), and receive appointment reminders. The system prevents double-booking through availability slot management.

4) Medicine Refill Ordering: Patients can order medication refills through integrated pharmacies. The order lifecycle follows a state machine: `PENDING` → `CONFIRMED` → `PREPARING` → `SHIPPED` → `DELIVERED`, with server-side validation ensuring only valid transitions occur.

5) Health Insights Dashboard: Patients access a risk dashboard displaying their composite health risk score (0–100), vital sign trends, personalized health suggestions, and active alerts—all computed on-device by the analytics pipeline described in Section V.

C. Doctor Workflows

1) Patient Management: Doctors view a list of assigned patients with key indicators including latest vitals, adherence

rate, and risk category. Doctors can access patient health logs, medication lists, and intake history.

2) Clinical Decision Support: Doctors create clinical decisions spanning seven types: *Prescription*, *Follow-Up*, *Vital Alert*, *Recommendation*, *Diagnosis*, *Lab Order*, and *Lifestyle*. Prescriptions include structured fields for medication name, dosage, frequency, route, unit, duration, and instructions, drawn from standardized medical reference lists.

3) Prescription Management: Prescriptions follow a versioned lifecycle (`ACTIVE` → `MODIFIED` → `STOPPED` → `COMPLETED`). Each modification generates a `VersionEntry` recording changed fields, previous values, change notes, and the modifying physician—ensuring a complete history of prescription changes.

4) Doctor Notes: Physicians can create public notes (visible to the patient) and private notes (visible only to the authoring doctor and administrators), supporting both patient communication and internal clinical documentation.

D. Pharmacy Workflows

1) Inventory Management: Pharmacies manage their medicine inventory with real-time stock tracking, including stock quantities, unit prices, expiry dates, batch numbers, and low-stock thresholds. Inventory items use a soft-delete pattern (`isActive` flag) to preserve historical references.

2) Order Fulfillment: Pharmacies receive refill orders and advance them through the order state machine. The `VerifyAndAcceptOrderUseCase` handles prescription verification for controlled medications before allowing order acceptance.

3) Transaction History: All completed order transactions are recorded immutably, including the amount, prescription verification status, and associated order and pharmacy identifiers.

E. Administrator Workflows

Administrators perform user management (approve/reject doctor and pharmacy accounts), monitor system activity through audit logs, and have read access to all orders. Critically, the `ADMIN` role cannot be created through client SDKs—the Firestore rules restrict user creation to `PATIENT`, `DOCTOR`, and `PHARMACY` roles, requiring server-side Firebase Admin SDK operations for admin account provisioning.

V. ON-DEVICE HEALTH ANALYTICS PIPELINE

A. Pipeline Overview

MediTrack’s analytics pipeline is a central contribution of this work. Unlike cloud-dependent analytics systems, the pipeline executes entirely on-device, providing three key advantages: (i) zero-latency health insights, (ii) privacy preservation by keeping sensitive health data local during computation, and (iii) offline availability. The pipeline is orchestrated by the `InsightEngine` facade, which coordinates five specialized engines. Fig. 4 illustrates the pipeline architecture.

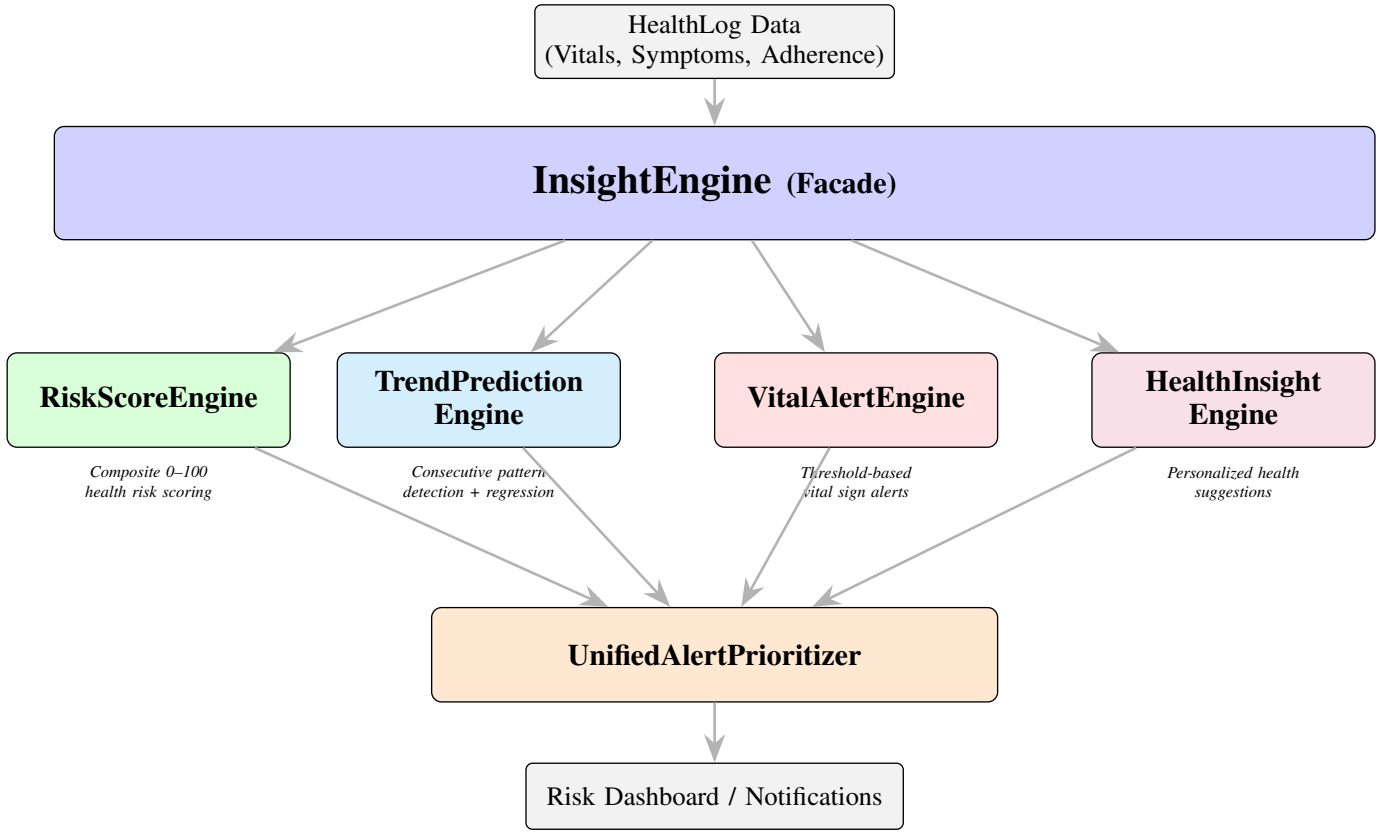


Fig. 4. On-device health analytics pipeline architecture. The *InsightEngine* facade coordinates the *RiskScoreEngine* (composite 0–100 scoring), *TrendPredictionEngine* (consecutive pattern detection with linear regression), and *VitalAlertEngine* (threshold-based vital alerts). Outputs are unified by the *UnifiedAlertPrioritizer* and fed to the *HealthInsightEngine* for personalized suggestion generation.

TABLE III
RISK SCORE DIMENSION WEIGHTS

Dimension	Wt.	Description
Vital Signs (BP, HR, Glucose, Temp)	40%	Clinical thresholds per medical guidelines
Medication Adherence	25%	Inverted adherence % (100% = 0 risk)
Symptom Burden	20%	Severity-weighted (severe=25, mod.=12, mild=5)
Data Recency & Completeness	15%	Days since last log, weekly frequency

B. RiskScoreEngine: Composite Health Risk Scoring

The *RiskScoreEngine* computes a composite health risk score ranging from 0 to 100 through weighted multi-dimensional analysis. The score integrates four dimensions, each reflecting a distinct aspect of patient health, as shown in Table III.

The Vital Signs dimension evaluates blood pressure, heart rate, glucose, and temperature against clinically established thresholds. Blood pressure scoring follows the Joint National Committee (JNC) staging guidelines as shown in Table IV.

Glucose scoring follows American Diabetes Association (ADA) thresholds, with severe hyperglycemia (≥ 300 mg/dL) scoring 100, very high (≥ 200) scoring 85, diabetic range

TABLE IV
BLOOD PRESSURE RISK SCORING THRESHOLDS

Sys. (mmHg)	Dia. (mmHg)	Score	Clinical Stage
≥ 180	≥ 120	100	Hypertensive Crisis
≥ 160	≥ 100	85	Stage 2+ Hypertension
≥ 140	≥ 90	70	Stage 1 Hypertension
≥ 130	≥ 85	50	Elevated
80–129	50–84	0–30	Normal Range
< 80	< 50	80	Dangerously Low

(≥ 126) scoring 60, pre-diabetic (100–125) scoring 30, hypoglycemia (< 70) scoring 75, and severe hypoglycemia (< 54) scoring 95.

The engine applies four multi-factor correlation rules that add bonus risk points when multiple adverse factors co-occur:

- *Metabolic Syndrome Risk* (+10 points): High BP + elevated glucose + poor adherence.
- *Cardiac Stress* (+12 points): Elevated BP + elevated HR + chest pain symptoms.
- *Uncontrolled Diabetic Risk* (+8 points): High glucose + poor adherence.
- *Symptomatic Non-Adherence* (+5 points): Recurring symptoms + missed medications.

TABLE V
VITAL ALERT ENGINE THRESHOLD VALUES

Vital Sign	Crit. High	Warn. High	Warn. Low	Crit. Low
BP (mmHg)	$\geq 180/120$	$\geq 140/90$	$< 90/60$	$< 80/50$
Glucose	≥ 300	≥ 200	< 70	< 54
HR (bpm)	≥ 150	≥ 120	< 60	< 40
Temp ($^{\circ}\text{C}$)	≥ 39.5	≥ 38.0	< 36.0	< 35.0

The composite risk score R is computed as:

$$R = \min \left(100, \sum_{i=1}^4 w_i \cdot d_i + \sum_{j=1}^4 c_j \right) \quad (1)$$

where w_i and d_i represent the weight and normalized score (0–100) of each dimension, and c_j represents the correlation bonus for each multi-factor rule. The final score is categorized into four risk levels: LOW (0–25), MODERATE (26–50), HIGH (51–75), and CRITICAL (76–100).

C. TrendPredictionEngine: Consecutive Pattern Detection

The TrendPredictionEngine detects directional trends in six vital sign types: heart rate, systolic blood pressure, diastolic blood pressure, glucose, weight, and temperature. For each vital type, the engine examines chronologically ordered readings and counts consecutive increases or decreases. A change exceeding 1.5% between adjacent readings qualifies as directional. Three or more consecutive directional changes trigger a trend alert; five or more trigger an accelerating trend alert.

When a trend is detected, a simple linear regression model is fit to the trending values to project the next expected reading. Given n trending values $(x_1, y_1), \dots, (x_n, y_n)$, the projected value is:

$$\hat{y}_{n+1} = \bar{y} + b(x_{n+1} - \bar{x}) \quad (2)$$

where $b = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$ is the regression slope.

This approach deliberately avoids complex machine learning models in favor of interpretable, deterministic algorithms that can be validated against clinical intuition and operate without training data.

D. VitalAlertEngine: Real-Time Threshold Alerting

The VitalAlertEngine evaluates individual health log readings against clinically informed thresholds to generate immediate alerts. The engine classifies each reading into one of three severity levels: NORMAL, WARNING, or CRITICAL. Table V presents the threshold values for each vital type.

E. HealthInsightEngine: Personalized Health Suggestions

The HealthInsightEngine generates up to eight prioritized, actionable health suggestions by synthesizing outputs from the other engines. Suggestions are categorized by action

type: LIFESTYLE, MEDICATION, DOCTOR-VISIT, MONITORING, and EMERGENCY. Suggestions are drawn from six sources:

- 1) *Risk-based*: CRITICAL risk triggers “Seek Medical Attention”; HIGH risk triggers “Schedule a Doctor Visit.”
- 2) *Trend-based*: Rising vital patterns with specific behavioral recommendations.
- 3) *Adherence-based*: Adherence below 50% triggers high-priority warnings.
- 4) *Vital-specific*: Metabolic risk detection for concurrent high BP and glucose.
- 5) *Data quality*: Prompts for missing or stale health data.
- 6) *Positive reinforcement*: Congratulatory feedback for low risk with high adherence.

F. UnifiedAlertPrioritizer: Cross-Engine Alert Fusion

The UnifiedAlertPrioritizer addresses the challenge of heterogeneous alert formats across engines. It maps four distinct priority taxonomies (AlertSeverity, TrendPriority, FactorSeverity, SuggestionPriority) to a common six-level UnifiedPriority scale: INFORMATIONAL, LOW, MODERATE, HIGH, URGENT, and EMERGENCY. The prioritizer deduplicates alerts by identifier prefix (retaining the highest-severity instance) and sorts results in descending priority order.

G. Supporting Components

PredictiveAlertManager prevents alert fatigue by implementing a 24-hour debounce window per alert type using SharedPreferences. If the same alert condition persists across multiple analytics runs within the window, only the first instance generates a notification.

ReminderOptimizer analyzes medication intake patterns to suggest optimal reminder times. The algorithm compares actual intake timestamps against scheduled reminders, identifies time slots with less than 60% adherence, suggests meal-aligned alternatives (08:00, 13:00, 20:00) for better adherence, detects over-scheduling (more than 3 medications at the same time), and flags inconvenient timing (before 06:00 or after 23:00).

VI. SECURITY ARCHITECTURE

A. Defense-in-Depth Model

MediTrack implements a layered security architecture that addresses threats at multiple levels. Fig. 5 illustrates the security stack, comprising Application, Transport, and Server layers.

B. Firestore Security Rules: Server-Side RBAC

The Firestore security rules (540 lines) implement comprehensive role-based access control through 11 helper functions. Key security patterns include:

- 1) **Role Verification**: Each protected operation verifies both the user’s role and their approval status through composite helper functions such as `isApprovedDoctor()`, which combines authentication, role, and status checks.

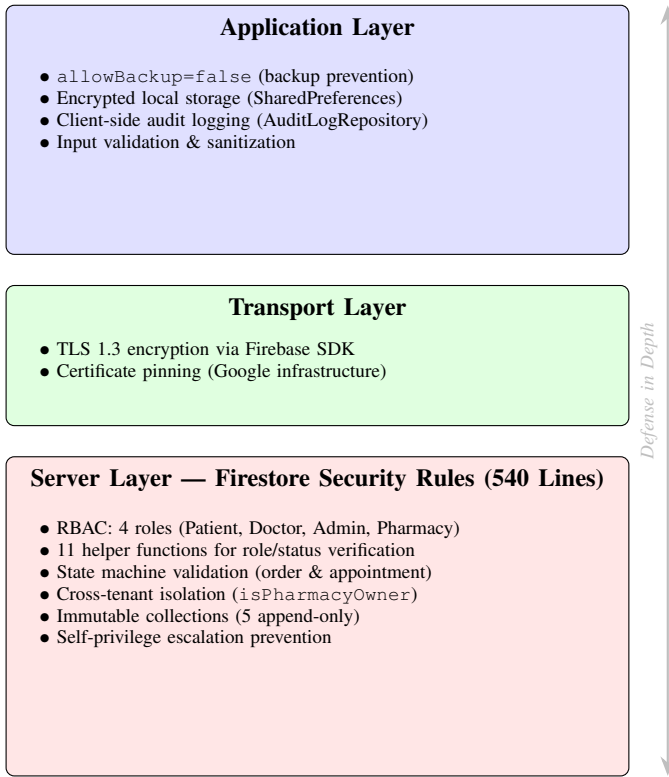


Fig. 5. MediTrack defense-in-depth security architecture. The Application Layer enforces backup prevention, encrypted local storage, and client-side audit logging. The Transport Layer provides TLS encryption via Firebase SDK. The Server Layer implements 540-line Firestore Security Rules with RBAC, state machine validation, immutable audit trails, and cross-tenant isolation.

2) Cross-Tenant Isolation: Pharmacy data is isolated through ownership verification via `isPharmacyOwner(pharmacyId)`, which cross-references the requesting user against the pharmacy’s `ownerId` field.

3) Doctor-Patient Assignment Verification: Access to patient clinical data requires an established doctor-patient relationship, verified through the `isDoctorAssignedToPatient(patientId)` function that checks membership in the patient’s `assignedDoctors` array.

4) Self-Privilege Escalation Prevention: Users cannot modify their own role or status after account creation, and the ADMIN role cannot be assigned through client SDKs.

C. Data Immutability Guarantees

Five collections enforce immutability at the server level, as summarized in Table VI.

D. State Machine Validation

Order and appointment status transitions are validated server-side, preventing invalid workflow jumps. For orders, valid transitions are: PENDING → CONFIRMED or CANCELLED, CONFIRMED → PREPARING or CANCELLED, PREPARING → SHIPPED, and SHIPPED → DELIVERED. This server-side enforcement prevents malicious clients from bypassing the intended workflow sequence.

TABLE VI
IMMUTABLE COLLECTION POLICIES

Collection	Policy	Rationale
prescriptions	Never deleted	Audit trail
medicineIntakes	No update/del.	Adherence integrity
auditLogs	Append-only	Tamper-proof logs
transactions	No update/del.	Financial integrity
riskScores	Append-only	Temporal tracking

TABLE VII
CODEBASE METRICS

Metric	Value
Total Kotlin source files	148
Total lines of Kotlin code	~26,753
Layout XML files	74
Activity classes	33
ViewModel classes	22
Repository classes	14
Data model classes	17
Analytics engine classes	9
Firestore collections	17
Security rule helper functions	11
Security rules (lines)	540
Domain use cases	3
Notification channels	3
User roles	4
Clinical decision types	7
Order status states	6
Min SDK / Target SDK	24 / 36

E. Data Validation

Input validation is enforced server-side for critical data types, including string length limits (email: 1–320 characters, display name: 1–100 characters, medication name: 1–200 characters), required field presence, and enum value constraints. This server-side validation complements client-side checks, ensuring data integrity even if the client application is modified.

VII. EVALUATION

A. Quantitative Codebase Metrics

Table VII presents quantitative metrics characterizing the MediTrack codebase, demonstrating the substantial engineering scope of the platform.

B. Architecture Quality Assessment

We evaluate MediTrack’s architecture against established software quality attributes:

1) Modularity: The MVVM architecture with dependency injection achieves high modularity. Each feature area (medicine, health logs, appointments, pharmacy) is encapsulated in dedicated packages with clear boundaries. The 14 repositories provide well-defined data access interfaces.

2) Separation of Concerns: The four-layer architecture (Presentation, Domain, Data, Infrastructure) ensures that UI logic, business rules, data access, and platform concerns remain decoupled. ViewModels contain no direct Firebase references, communicating exclusively through repository abstractions.

3) Security Coverage: Server-side security rules cover all 17 collections with role-specific access patterns. Five collections enforce immutability. Two collections implement state-machine validation. Client-side security includes backup prevention and encrypted local storage.

4) Offline Resilience: The dual offline strategy (100 MB Firestore PersistentCache plus SharedPreferences-based OfflineQueueManager) enables continued operation without network connectivity. WorkManager-based periodic synchronization at 15-minute intervals ensures eventual consistency.

C. Feature Coverage Analysis

Table VIII compares MediTrack’s feature coverage with the requirements identified in recent mHealth literature surveys [17], [18].

D. Analytics Pipeline Evaluation

To evaluate the analytics pipeline’s clinical alignment, we compared MediTrack’s threshold values against published clinical guidelines. Blood pressure thresholds align with AHA/ACC 2017 guidelines [19]. Glucose thresholds align with ADA 2024 Standards of Care [20]. Heart rate thresholds align with standard clinical ranges [21]. Temperature thresholds align with WHO fever classification [22]. The multi-factor correlation rules (metabolic syndrome risk, cardiac stress, uncontrolled diabetic risk) are based on established clinical associations documented in medical literature [23], [24].

VIII. DISCUSSION

A. Key Contributions

MediTrack demonstrates that a mobile-first, multi-stakeholder health platform can achieve comprehensive feature coverage while maintaining architectural quality and security rigor. Three aspects warrant further discussion.

On-device analytics as a viable alternative: The analytics pipeline’s deterministic, rule-based approach offers advantages over cloud-based ML: interpretability (physicians can understand why a risk score was assigned), reproducibility (same inputs always produce same outputs), and privacy (no health data leaves the device for analytics). While machine learning approaches may achieve higher predictive accuracy for specific tasks, the rule-based approach is more amenable to clinical validation and regulatory approval.

Server-side security as primary defense: By enforcing access control, data immutability, and workflow validation in Firestore Security Rules, MediTrack ensures that security cannot be bypassed through client-side manipulation—an approach aligned with the zero-trust security model. The 540-line rule set demonstrates that declarative security rules can express complex healthcare access patterns including multi-tenant isolation and state machine validation.

Unified stakeholder platform: The four-role architecture eliminates data silos between care participants. A prescription created by a doctor is immediately visible to the patient and the fulfilling pharmacy, with every status change audited and validated. This unified approach reduces the coordination overhead inherent in multi-application healthcare ecosystems.

B. Limitations

- 1) *No clinical trial validation:* While the analytics thresholds are sourced from established guidelines, the composite risk scoring algorithm has not been validated through a clinical trial. Future work should conduct a prospective study comparing MediTrack’s risk assessments against clinician judgments.
- 2) *Limited interoperability:* MediTrack uses a Firebase-specific data model without HL7 FHIR or openEHR support, limiting integration with existing healthcare IT infrastructure.
- 3) *No end-to-end encryption for chat:* While messages are encrypted in transit (TLS) and at rest (Firestore server-side encryption), the platform does not implement end-to-end encryption for doctor-patient communication.
- 4) *Single-platform implementation:* The current implementation targets Android only. A cross-platform approach using Kotlin Multiplatform or Flutter would broaden accessibility.
- 5) *Rule-based analytics limitations:* The deterministic analytics pipeline cannot learn from individual patient patterns or population-level data. Integration with federated learning or on-device ML models could enhance personalization while preserving privacy.

C. Future Work

Based on the identified limitations, we outline the following directions:

- 1) **Clinical validation study:** Conduct a multi-site prospective study to validate the composite risk score against clinician assessments and patient outcomes.
- 2) **HL7 FHIR integration:** Implement FHIR resource mappings for interoperability with hospital EHR systems and health information exchanges.
- 3) **Federated learning:** Integrate on-device federated learning to improve analytics personalization without centralizing patient data.
- 4) **Wearable device integration:** Extend the health log input pipeline to accept continuous data streams from wearables via the Health Connect API.
- 5) **Cloud Functions:** Migrate critical business logic (prescription verification, order processing) to Firebase Cloud Functions for additional server-side enforcement.
- 6) **Cross-platform expansion:** Extend the platform to iOS using Kotlin Multiplatform for shared business logic.

IX. CONCLUSION

This paper presented MediTrack, a multi-stakeholder mobile health platform that addresses the critical gap between fragmented, single-role mHealth applications and the integrated needs of healthcare ecosystems. Through its four-role architecture (Patient, Doctor, Pharmacy, Administrator), MediTrack provides a unified platform for medication management, health monitoring, clinical decision support, pharmacy operations, and secure communication.

The on-device health analytics pipeline—comprising RiskScoreEngine, TrendPredictionEngine,

TABLE VIII
FEATURE COVERAGE VS. MHEALTH REQUIREMENTS

Requirement	Feature	MediTrack Implementation
Medication Mgmt.	Reminder scheduling Adherence tracking Low stock alerts Refill ordering	ExactAlarm + ForegroundService Immutable intake records Threshold-based detection State machine workflow
Health Monitoring	Vital sign logging Trend detection Risk assessment Threshold alerts	6 vital types + symptoms Consecutive + lin. regression Composite 0–100 (4 dims.) Evidence-based thresholds
Clinical Support	Prescription mgmt. Clinical decisions Doctor notes	Versioned, immutable records 7 decision types Public + private
Communication	Doctor-patient chat Read receipts Appt. scheduling	Real-time Firestore messaging Per-message tracking 4 appointment types
Pharmacy	Inventory mgmt. Order fulfillment Rx verification	Real-time, soft-delete pattern 6-state machine Domain use case driven
Security	Authentication Authorization Audit trail	Email/password + Google OAuth 4 roles, server-side RBAC Immutable, append-only
Offline Support	Offline data access Background sync	100 MB persistent cache WorkManager, 15-min interval

VitalAlertEngine, HealthInsightEngine, and UnifiedAlertPrioritizer—demonstrates that clinically meaningful health intelligence can be delivered without cloud computation dependencies, preserving patient privacy and enabling offline operation. The composite risk score, computed through weighted multi-dimensional analysis with multi-factor correlation rules as expressed in (1), provides patients and clinicians with an interpretable, evidence-based health assessment.

The defense-in-depth security model, anchored by 540 lines of server-side Firestore Security Rules, ensures data integrity through role-based access control, immutable audit trails, and state-machine-validated workflows. The security architecture demonstrates that healthcare-grade access control can be implemented using declarative server-side rules without requiring custom backend infrastructure.

With 148 source files comprising approximately 26,753 lines of Kotlin code, 17 Firestore collections, and comprehensive coverage of mHealth requirements identified in the literature, MediTrack represents a substantial engineering contribution to the mobile health domain. The platform's architecture provides a reference implementation for future multi-stakeholder health applications seeking to balance clinical functionality, security rigor, and mobile-first design.

ACKNOWLEDGMENT

The authors would like to thank the Department of Computer Science and Engineering, Narayana Engineering College, Nellore, for providing the infrastructure and academic support for this research. Special thanks to the faculty members

who provided valuable feedback during the development and evaluation of MediTrack.

REFERENCES

- [1] Grand View Research, "mHealth Market Size, Share & Trends Analysis Report, 2024–2032," 2024.
- [2] Medisafe, "Medisafe Medication Management Platform." [Online]. Available: <https://www.medisafe.com>
- [3] MyTherapy, "MyTherapy: Medication Reminder & Health Tracker." [Online]. Available: <https://www.mytherapyapp.com>
- [4] CareZone, "CareZone Medication Management." [Online]. Available: <https://carezone.com>
- [5] Epic Systems, "MyChart Patient Portal." [Online]. Available: <https://www.mychart.com>
- [6] Apple Inc., "Apple Health." [Online]. Available: <https://www.apple.com/health>
- [7] Google LLC, "Health Connect." [Online]. Available: <https://developer.android.com/health-connect>
- [8] B. Middleton, D. F. Sittig, and A. Wright, "Clinical Decision Support: a 25 Year Retrospective and a 25 Year Vision," *Yearbook of Medical Informatics*, vol. 25, no. S 01, pp. S25–S30, 2016.
- [9] A. X. Garg *et al.*, "Effects of Computerized Clinical Decision Support Systems on Practitioner Performance and Patient Outcomes: A Systematic Review," *JAMA*, vol. 293, no. 10, pp. 1223–1238, 2005.
- [10] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," *Nature Medicine*, vol. 25, no. 1, pp. 44–56, 2019.
- [11] K. Chen *et al.*, "HealthAssist: A Mobile Health Application for Activity-Based Health Predictions," *IEEE J. Biomed. Health Inform.*, vol. 25, no. 4, pp. 1087–1096, 2021.
- [12] VitalConnect Inc., "VitalConnect Continuous Monitoring Platform." [Online]. Available: <https://vitalconnect.com>
- [13] L. H. Iwaya *et al.*, "Security and Privacy for mHealth and uHealth Systems: A Systematic Mapping Study," *IEEE Access*, vol. 8, pp. 150081–150112, 2020.
- [14] OWASP Foundation, "OWASP Mobile Security Testing Guide." [Online]. Available: <https://owasp.org/www-project-mobile-security-testing-guide>

- [15] Google LLC, "Firebase Security Rules Documentation." [Online]. Available: <https://firebase.google.com/docs/rules>
- [16] S. Sudhodanan *et al.*, "Security Analysis of Firebase-backed Mobile Applications," in *Proc. ACM SIGSAC Conf. Computer and Communications Security*, 2020.
- [17] C. Free *et al.*, "The Effectiveness of Mobile-Health Technology-Based Health Behaviour Change or Disease Management Interventions for Health Care Consumers: A Systematic Review," *PLoS Medicine*, vol. 10, no. 1, e1001362, 2013.
- [18] J. Batista *et al.*, "Requirements for Mobile Health Applications: A Systematic Literature Review," *J. Med. Syst.*, vol. 44, no. 5, pp. 1–13, 2020.
- [19] P. K. Whelton *et al.*, "2017 ACC/AHA/... High Blood Pressure Guidelines," *J. Am. Coll. Cardiol.*, vol. 71, no. 19, pp. e127–e248, 2018.
- [20] American Diabetes Association, "Standards of Care in Diabetes—2024," *Diabetes Care*, vol. 47, Suppl. 1, 2024.
- [21] G. D. Perkins *et al.*, "European Resuscitation Council Guidelines 2021," *Resuscitation*, vol. 161, pp. 1–60, 2021.
- [22] World Health Organization, "Integrated Management of Childhood Illness Chart Booklet," WHO Press, 2014.
- [23] S. M. Grundy *et al.*, "Definition of Metabolic Syndrome," *Circulation*, vol. 109, no. 3, pp. 433–438, 2004.
- [24] C. P. Cannon, "Cardiovascular Disease and Modifiable Cardiometabolic Risk Factors," *Clin. Cornerstone*, vol. 8, no. 3, pp. 11–28, 2007.

Vijaya Wujjwal Reddy is currently pursuing a B.Tech degree in Computer Science and Engineering at Narayana Engineering College, Nellore, India. His research interests include mobile application development, mobile health (mHealth) systems, cloud computing with Firebase, health analytics, and software architecture design. He is the primary developer of the MediTrack platform presented in this paper, responsible for the overall system architecture, MVVM design pattern implementation, on-device clinical analytics pipeline comprising four specialized engines, role-based security framework with 540 lines of Firestore Security Rules, and end-to-end Firebase backend integration spanning 17 Firestore collections and 14 repository classes. He has hands-on experience in Kotlin, Android Jetpack components, Hilt dependency injection, Google Maps SDK, Firebase Cloud Messaging, and real-time NoSQL database systems.

Baddipudi Krishna Sai is currently pursuing a B.Tech degree in Computer Science and Engineering at Narayana Engineering College, Nellore, India. His research interests include mobile application development, database management, cloud-based storage systems, and healthcare informatics. He contributed to the MediTrack platform in the areas of Firestore database schema design, entity-relationship data modeling across 17 collections, pharmacy inventory management module development, medicine catalog integration, and seed data generation for testing and validation. He has experience in NoSQL database design, Android development, data normalization techniques for healthcare systems, and automated testing methodologies.

Budamagunta Ravi Teja is currently pursuing a B.Tech degree in Computer Science and Engineering at Narayana Engineering College, Nellore, India. His research interests include software engineering, cloud computing, real-time systems, and mobile health platforms. He contributed to the MediTrack platform in the areas of order tracking workflow implementation with state-machine validation across six order states, real-time push notification system using Firebase Cloud Messaging with token management, and map-based pharmacy discovery with Google Maps API integration and distance-based sorting algorithms. He has experience in location-based services, event-driven architectures, Android foreground services, and broadcast receiver design patterns.

Nuthalapati Lohith is currently pursuing a B.Tech degree in Computer Science and Engineering at Narayana Engineering College, Nellore, India. His research interests include software development, user interface design, human-computer interaction, and health information technology. He contributed to the MediTrack platform in the areas of UI/UX design following Material Design 3 guidelines, patient home dashboard development with role-specific navigation, health log management with vital sign tracking and trend visualization, and medication reminder interface implementation with exact alarm scheduling via Android AlarmManager. He has experience in Android XML layouts, responsive design patterns, RecyclerView adapters, and accessibility-focused mobile application development.

K. Divya M.Tech (Ph.D), is an Assistant Professor in the Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India. Her research interests include software engineering, mobile computing, machine learning, healthcare informatics, and secure system design. She served as the project mentor and guided the architectural design, evaluation methodology, and defense-in-depth security framework of the MediTrack platform. She provided critical insights on clinical decision support system design, multi-stakeholder role-based access control modeling, state-machine-validated workflows, and compliance with healthcare data protection standards. She has published research in the areas of software quality assurance, intelligent computing systems, and mobile application security frameworks.